

Linear Regression and XGBoost Comparison I

Elliot Dean
Office of the Chief Financial Officer | Office of the Assessor
City of Detroit

19 August 2025

Contents

Load Necessary Libraries	1
Custom Functions	1
Load Sales Data	1
Model Parameters Setup and Training	1
Graphical Results	4
Save Models	8

Load Necessary Libraries

```
library(tidyverse)  # data wrangling & ggplot2
library(magrittr)
library(janitor)
library(kableExtra) # Create unique tables
library(caret)      # train/test split, cross-validation
library(broom)      # tidy model output
library(car)         # VIF, Durbin-Watson
library(lmtest)      # Breusch-Pagan test
library(tidymodels) # collection of packages for modeling and machine learning
library(xgboost)     # Extreme Gradient Boosting
library(bundle)      # Serialize Model Objects with a Consistent Interface
```

Custom Functions

```
# helper for pretty tables
tbl <- function(x, cap = NULL, digits = 4){
  knitr::kable(x, format = "latex", booktabs = TRUE, digits = digits, caption = cap) |>
    kable_styling(latex_options = c("hold_position"))
}
```

Load Sales Data

```
load('C:/Users/satur/GitHub/dOCF0modeling/R_Project/data/cod_23_25_lnprice_data.RData')
```

Model Parameters Setup and Training

```
# =====
# Compare Linear Regression vs XGBoost (tidymodels framework)
# =====

set.seed(123) # Randomized number generator

# ---- USER INPUTS ----
target      <- "LN_PRICE" # outcome column name (string)
predictors  <- c('CRAWL', 'SLAB', 'LN_LANDRATIO', 'ARTERIAL',
                'SINGLE_FAMILY1STORY', 'SINGLE_FAMILY2STORY', 'HALF_DUPLEX1STORY', 'HALF_DUPLEX2STORY',
                'SMALLEST', 'SMALL', 'LARGE', 'LARGEST', 'AVG',
                'TWO_PLUS_BATHS', 'POWDER_ROOM', 'LN_GARAGESPACES', 'FIREPLACE',
                'EXCELLENT', 'VERY_GOOD', 'GOOD', 'AVERAGE', 'FAIR', 'POOR', 'VERY_POOR_UNSOUND',
                'QABOVE_AVG', 'QBELOW_AVG', 'Q_AVG',
                'YB_PRE_1929', 'YB_1930_TO_1945', 'YB_1946_TO_1960', 'YB_POST_1961',
                'HOT_WATER', 'ELEC_BASEBOARD', 'ELEC_WALL', 'WALL', 'CENTRAL_AIR', 'ELEC_RADIANT',
                'RADIANT_FLOOR', 'SPACE_HEATER',
                'MONTHS_1TO9', 'MONTHS_10TO16', 'MONTHS_17TO24') # <- your chosen predictors

# -----

# (1) Keep only rows with a *finite* outcome (guards against -Inf/NaN after logs)
df <- df[is.finite(df[[target]]), , drop = FALSE]

# (2) Make a slim copy with only the target + chosen predictors
keep_cols <- intersect(colnames(df), c(target, predictors))
missing_in_df <- setdiff(c(target, predictors), keep_cols)
if (length(missing_in_df)) {
  stop("These columns are not in df: ", paste(missing_in_df, collapse = ", "))
}
df_small <- df[, keep_cols, drop = FALSE]

# ---- resampling ----
split <- initial_split(df_small, prop = 0.80, strata = !!sym(target))
train <- training(split)
test  <- testing(split)
folds <- vfold_cv(train, v = 10, strata = !!sym(target))

# ---- recipe (shared by both models) ----
# - remove any remaining NAs
# - drop zero-variance columns (per-fold safe)
# - one-hot encode factors
# - normalize numeric predictors
rec <- recipe(as.formula(paste(target, "~ .")), data = train) |>
  step_naomit(all_predictors(), all_outcomes()) |>
  step_zv(all_predictors()) |>
  step_dummy(all_nominal_predictors(), one_hot = TRUE) |>
  step_normalize(all_numeric_predictors())

# =====
# Model 1: Linear regression (no tuning)
# =====

spec_lm <- linear_reg() |>
  set_engine("lm")

wf_lm <- workflow() |>
  add_model(spec_lm) |>
```

```

add_recipe(rec)

# Cross-validated performance
lm_cv <- fit_resamples(
  wf_lm,
  resamples = folds,
  metrics   = metric_set(rmse, rsq, mae),
  control   = control_resamples(save_pred = TRUE)
)

# =====
# Model 2: XGBoost regression (tuned)
# =====
spec_xgb <- boost_tree(
  mode           = "regression",
  trees          = tune(),          # nrounds
  tree_depth     = tune(),
  learn_rate     = tune(),
  loss_reduction = tune(),          # gamma
  min_n          = tune(),          # min_child_weight (mapped)
  mtry           = tune(),          # colsample_bytree
  sample_size    = tune()          # subsample
) |>
  set_engine("xgboost")

wf_xgb <- workflow() |>
  add_model(spec_xgb) |>
  add_recipe(rec)

# Parameter grid (data-aware for mtry)
pset <- parameters(wf_xgb) |>
  # sensible bounds; adjust if needed
  update(
    trees          = trees(c(200, 1500)),
    tree_depth     = tree_depth(c(2, 10)),
    learn_rate     = learn_rate(c(-4, -1)), # on log10 scale; converted by dials
    loss_reduction = loss_reduction(c(-6, 3)),
    min_n          = min_n(c(1, 20)),
    mtry           = finalize(mtry(), train),
    sample_size    = sample_prop(c(0.5, 1.0))
  )

grid <- grid_latin_hypercube(pset, size = 30)

xgb_tuned <- tune_grid(
  wf_xgb,
  resamples = folds,
  grid      = grid,
  metrics   = metric_set(rmse, rsq, mae),
  control   = control_grid(save_pred = TRUE)
)

best_xgb <- select_best(xgb_tuned, metric = "rmse")
xgb_final_wf <- finalize_workflow(wf_xgb, best_xgb)

# =====
# Test-set evaluation (fair holdout)
# =====

```

```
# Linear: fit to full training set, evaluate on test
lm_last <- last_fit(wf_lm, split, metrics = metric_set(rmse, rsq, mae))
xgb_last <- last_fit(xgb_final_wf, split, metrics = metric_set(rmse, rsq, mae))
```

Graphical Results

```
# Cross-validated means (and SEs) per model
cv_lm_tbl <- collect_metrics(lm_cv)
cv_xgb_tbl <- collect_metrics(xgb_tuned) |>
  dplyr::filter(.config == best_xgb$.config)

# Side-by-side CV summary
cv_compare <- dplyr::bind_rows(
  cv_lm_tbl |> dplyr::mutate(model = "linear"),
  cv_xgb_tbl |> dplyr::mutate(model = "xgboost")
) |>
  dplyr::select(model, .metric, mean, std_err) |>
  tidyr::pivot_wider(names_from = model, values_from = c(mean, std_err))

tbl(cv_compare, cap = "10-fold CV metrics: Linear vs XGBoost")
```

Table 1: 10-fold CV metrics: Linear vs XGBoost

.metric	mean_linear	mean_xgboost	std_err_linear	std_err_xgboost
mae	0.4198	0.4289	0.0022	0.0024
rmse	0.5195	0.5349	0.0024	0.0029
rsq	0.3295	0.3222	0.0097	0.0066

```
# Per-fold paired deltas (xgb - linear)
lm_fold <- collect_metrics(lm_cv, summarize = FALSE)
xg_fold <- collect_metrics(xgb_tuned, summarize = FALSE)

paired <- dplyr::left_join(
  lm_fold, xg_fold,
  by = c("id", ".metric", ".estimator"),
  suffix = c("_lm", "_xgb")
) |>
  dplyr::mutate(delta = .estimate_xgb - .estimate_lm) |>
  dplyr::select(id, .metric, delta)

delta_summary <- paired |>
  dplyr::group_by(.metric) |>
  dplyr::summarise(mean_delta = mean(delta), sd_delta = sd(delta), .groups = "drop")

tbl(delta_summary, cap = "Per-fold mean delta (XGB - Linear)")
```

```
# Test-set metrics (holdout)
lm_test <- collect_metrics(lm_last) |> dplyr::mutate(model = "linear")
xgb_test <- collect_metrics(xgb_last) |> dplyr::mutate(model = "xgboost")

test_compare <- dplyr::bind_rows(lm_test, xgb_test) |>
  dplyr::select(model, .metric, .estimate) |>
```

Table 2: Per-fold mean delta (XGB – Linear)

.metric	mean_delta	sd_delta
mae	2.8632	3.6590
rmse	2.8447	3.6191
rsq	NA	NA

```
tidyr::pivot_wider(names_from = model, values_from = .estimate)

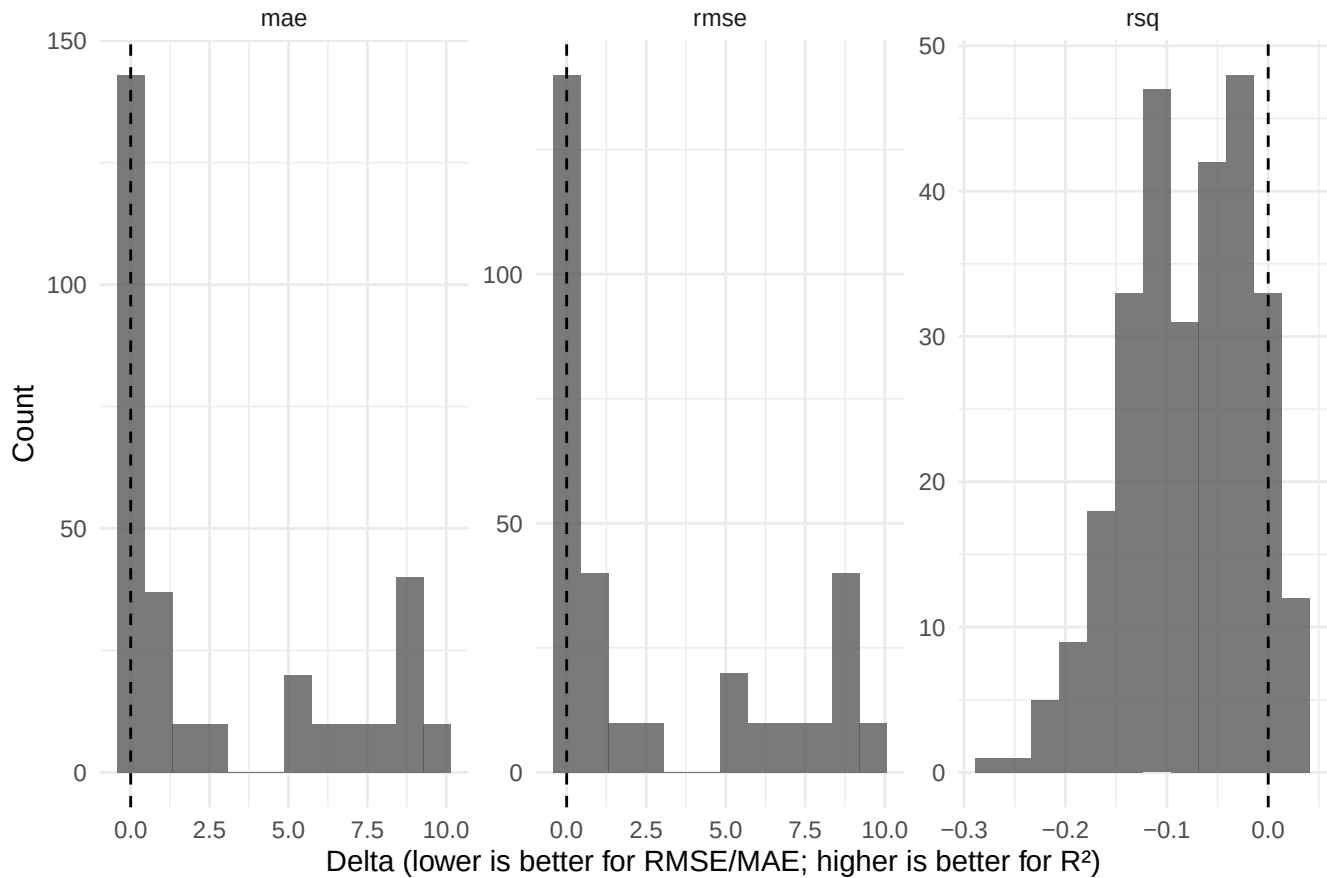
tbl(test_compare, cap = "Test-set metrics (holdout)")
```

Table 3: Test-set metrics (holdout)

.metric	linear	xgboost
rmse	0.5129	0.5246
rsq	0.3583	0.3678
mae	0.4131	0.4217

```
# 1) Distribution of per-fold deltas (xgb - linear)
ggplot(paired, aes(x = delta)) +
  geom_histogram(bins = 12, alpha = 0.8) +
  facet_wrap(~ .metric, scales = "free") +
  geom_vline(xintercept = 0, linetype = 2) +
  labs(title = "Per-fold performance deltas (XGB - Linear)",
       x = "Delta (lower is better for RMSE/MAE; higher is better for R²)",
       y = "Count") +
  theme_minimal()
```

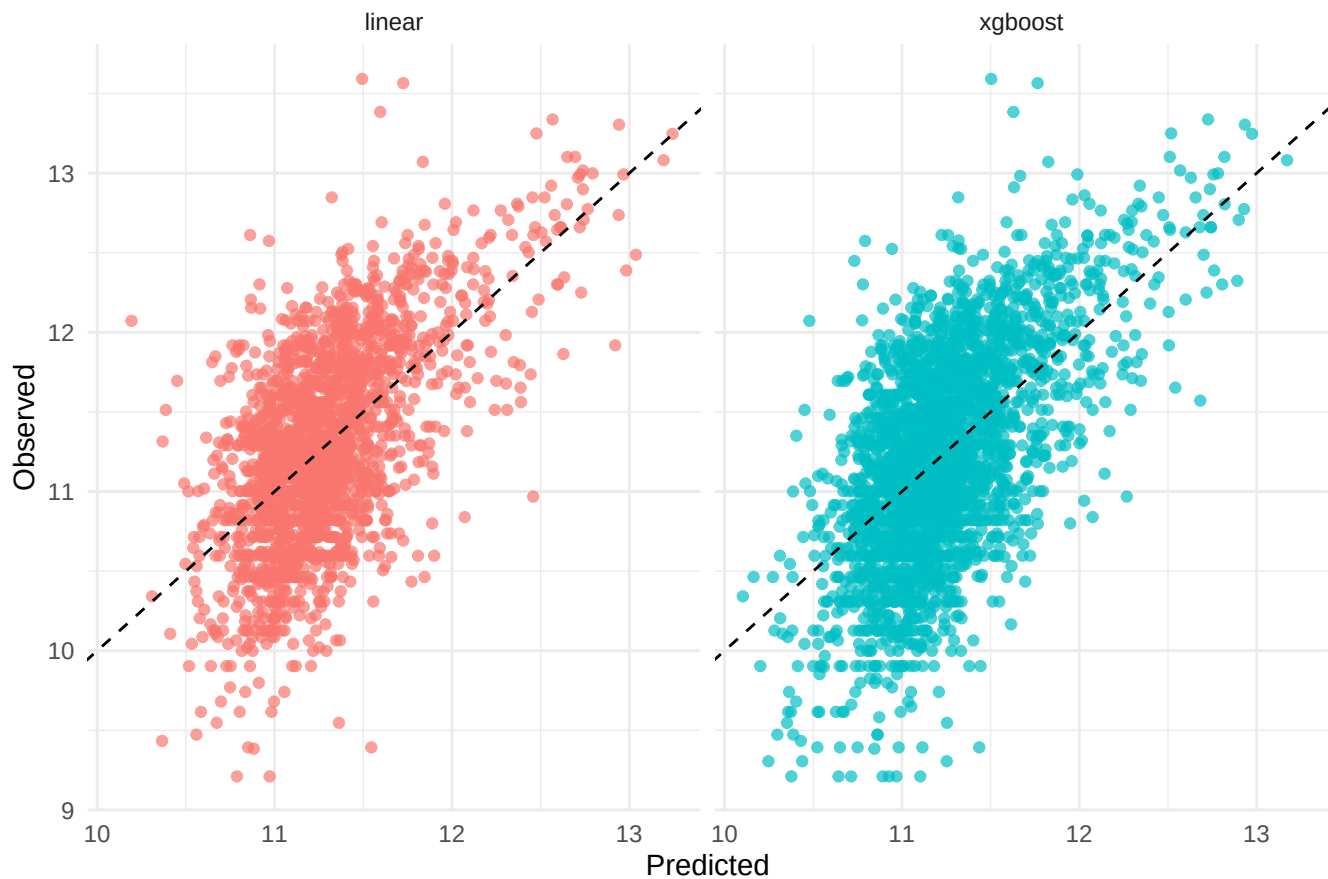
Per-fold performance deltas (XGB – Linear)



```
# 2) Test-set predictions: observed vs predicted (both models)
lm_pred  <- collect_predictions(lm_last)  |> dplyr::mutate(model = "linear")
xgb_pred  <- collect_predictions(xgb_last) |> dplyr::mutate(model = "xgboost")
pred_both <- dplyr::bind_rows(lm_pred, xgb_pred)

ggplot(pred_both, aes(x = .pred, y = !!sym(target), color = model)) +
  geom_point(alpha = 0.7) +
  geom_abline(slope = 1, intercept = 0, linetype = 2) +
  facet_wrap(~ model) +
  labs(title = "Test-set: Predicted vs Observed",
       x = "Predicted", y = "Observed") +
  theme_minimal() + theme(legend.position = "none")
```

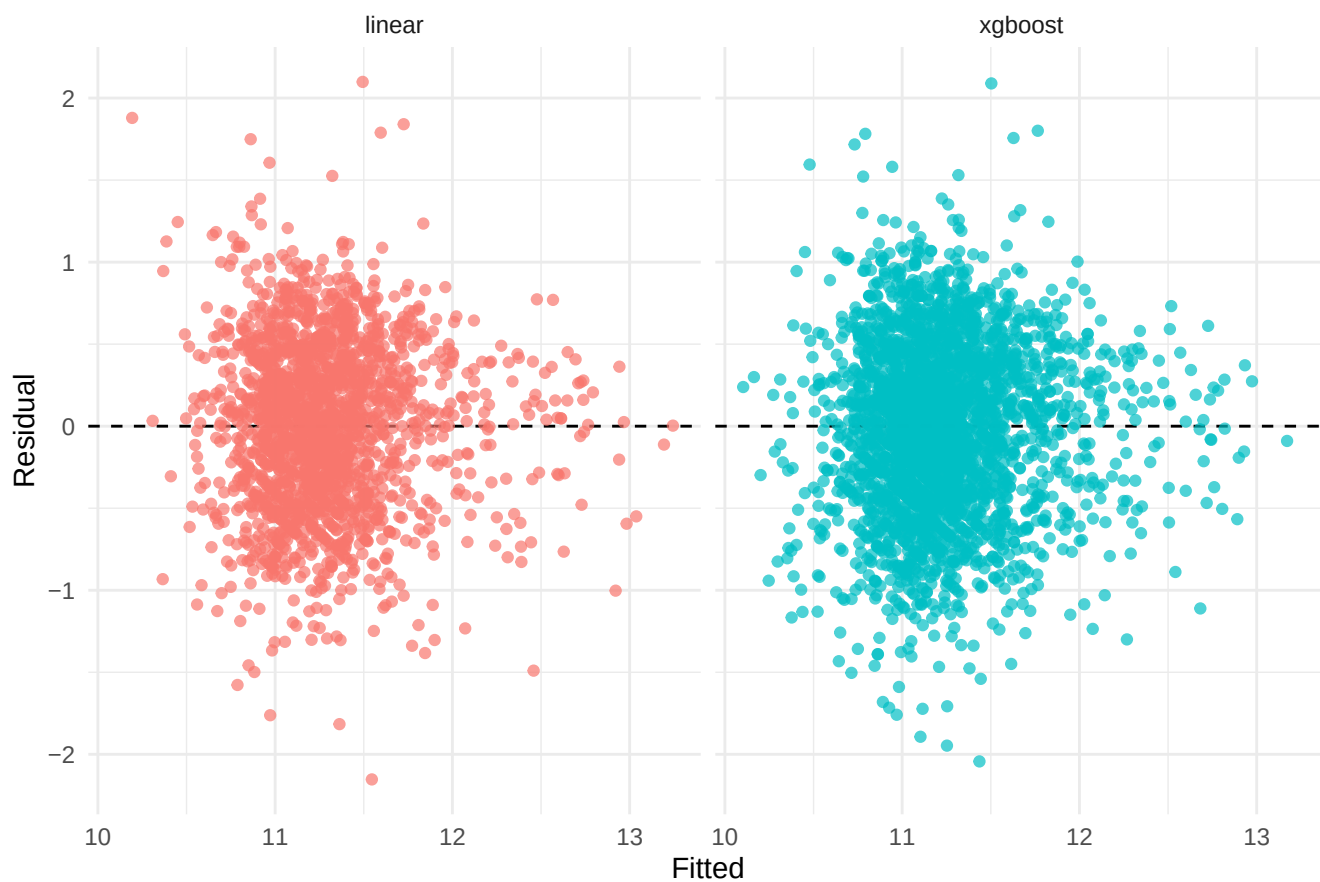
Test-set: Predicted vs Observed



```
# 3) Residuals vs fitted (test set)
pred_both <- pred_both |>
  dplyr::mutate(resid = !!sym(target) - .pred)

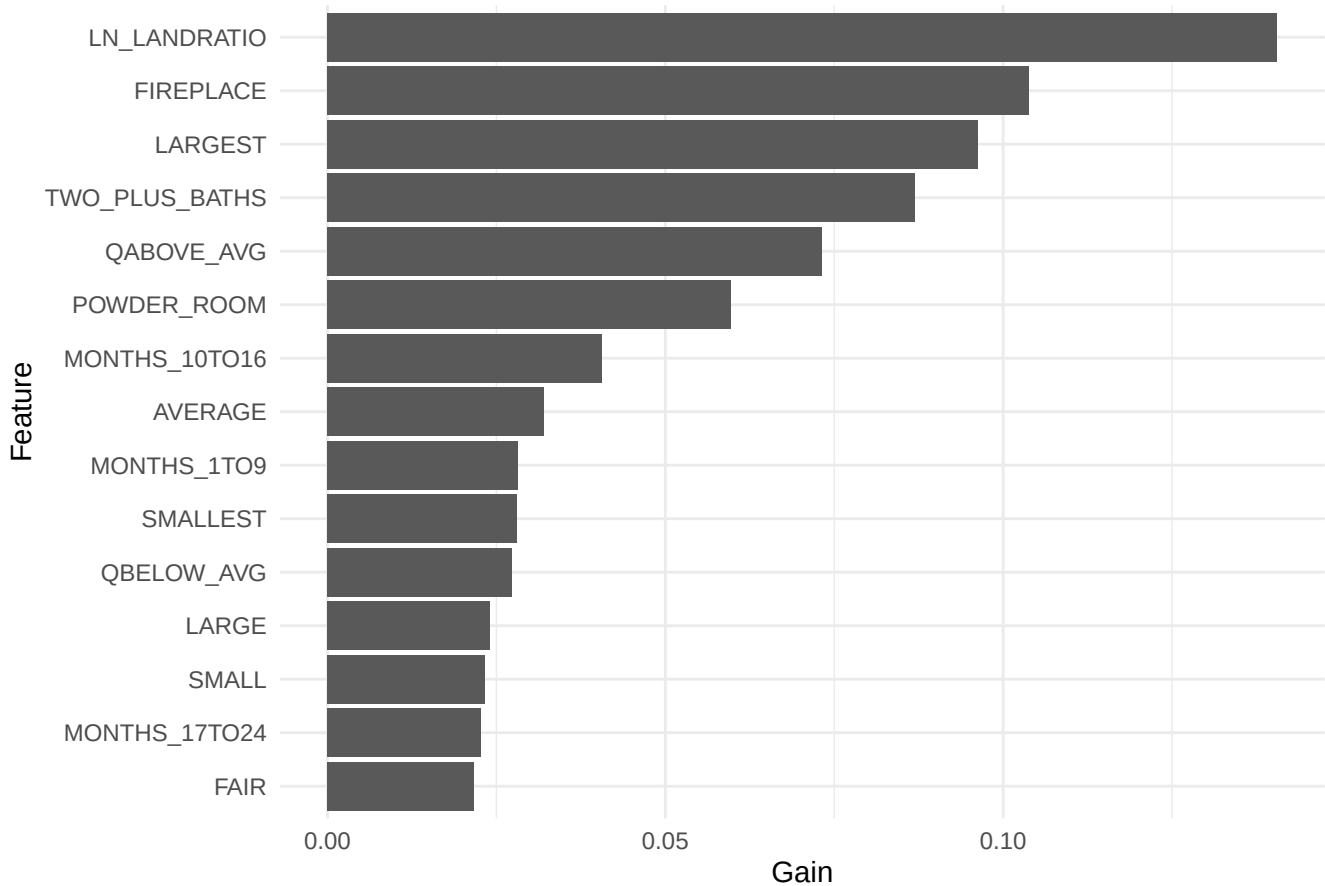
ggplot(pred_both, aes(x = .pred, y = resid, color = model)) +
  geom_hline(yintercept = 0, linetype = 2) +
  geom_point(alpha = 0.7) +
  facet_wrap(~ model) +
  labs(title = "Test-set Residuals vs Fitted",
       x = "Fitted", y = "Residual") +
  theme_minimal() + theme(legend.position = "none")
```

Test-set Residuals vs Fitted



```
# 4) XGBoost feature importance (Gain) - top 15
xgb_fit_obj <- extract_fit_parsnip(xgb_last$.workflow[[1]])$fit
if (inherits(xgb_fit_obj, "xgb.Booster")) {
  imp <- xgboost::xgb.importance(model = xgb_fit_obj)
  imp <- imp[1:min(15, nrow(imp)), ]
  ggplot(imp, aes(x = reorder(Feature, Gain), y = Gain)) +
    geom_col() +
    coord_flip() +
    labs(title = "XGBoost Feature Importance (Gain)", x = "Feature", y = "Gain") +
    theme_minimal()
}
```


XGBoost Feature Importance (Gain)



Save Models

```
# pull the already-fitted workflows out of last_fit tibbles
lm_wf_fitted <- lm_last$.workflow[[1]]
xgb_wf_fitted <- xgb_last$.workflow[[1]]

saveRDS(lm_wf_fitted,
        "C:/Users/satur/GitHub/dOCF0modeling/R_Project/models/lm_comp1_lnprice_workflow.rds")
saveRDS(xgb_wf_fitted,
        "C:/Users/satur/GitHub/dOCF0modeling/R_Project/models/xgb_comp1_lnprice_workflow.rds")
```

Console Output

```
# --- Cross-validated metrics (train folds) ---
cat("\n=== Cross-validated metrics (train folds) ===\n")
cv_lm_tbl <- collect_metrics(lm_cv)
print(cv_lm_tbl)

cat("\n--- XGBoost tuned (CV) ---\n")
cv_xgb_tbl <- collect_metrics(xgb_tuned) |>
  dplyr::filter(.config == best_xgb$.config)
print(cv_xgb_tbl)

# --- Test-set metrics (holdout) ---
```

```

cat("\n=== Test-set metrics ===\n")

cat("\nLinear model on test:\n")
lm_test_tbl <- collect_metrics(lm_last) # columns: .metric, .estimator, .estimate
print(lm_test_tbl)

cat("\nXGBoost (tuned) on test:\n")
xgb_test_tbl <- collect_metrics(xgb_last) # same columns
print(xgb_test_tbl)

# Side-by-side comparison for the test set
test_compare <- dplyr::bind_rows(
  lm_test_tbl |> dplyr::mutate(model = "linear"),
  xgb_test_tbl |> dplyr::mutate(model = "xgboost")
) |>
  dplyr::select(model, .metric, .estimate) |>
  tidyr::pivot_wider(names_from = model, values_from = .estimate)

cat("\n=== Test-set comparison (wide) ===\n")
print(test_compare)

# =====
# Inspect final fits / coefficients (optional)
# =====
# Linear coefficients
lm_fit_obj <- extract_fit_parsnip(lm_last$.workflow[[1]])$fit
cat("\nLinear coefficients (test fit):\n")
print(coef(lm_fit_obj))

# XGBoost importances
xgb_fit_obj <- extract_fit_parsnip(xgb_last$.workflow[[1]])$fit
if (inherits(xgb_fit_obj, "xgb.Booster")) {
  cat("\nXGBoost feature importance (Gain):\n")
  print(xgboost::xgb.importance(model = xgb_fit_obj)[1:10, ])
}

```